

Realistic or Generated Code

Background

There exist several tools that can automatically edit code for different purposes. They can be used to convert real-world normal code into vulnerable code. The generated vulnerable code samples may be similar to the realistic vulnerable code samples which are written by real developers. It may not be easy to differentiate the generated code from the realistic code in a short time.

Prerequisite

The participants must be familiar with the common vulnerability types in C language, knowing [Common Weakness Enumeration \(CWE\)](#) and [Common Vulnerabilities and Exposures \(CVE\)](#).

This Survey

In this survey, you will be asked to help us differentiate the generated code from the realistic code. **Given a vulnerable code sample, we will tell you what and where the vulnerability is in the code sample and show you how to fix the vulnerability. The vulnerability in the code sample may be injected by a code editing tool or originally caused by a real developer. You will be asked to tell your confidence that the vulnerability in the code sample is made by a real developer rather than a code editing tool.**

There are 4 levels of the confidence, where:

- 1: Strongly confident that the vulnerability in the code is made by a code editing tool
- 2: Weakly confident that the vulnerability in the code is made by a code editing tool
3. Weakly confident that the vulnerability in the code is made by a real developer
4. Strongly confident that the vulnerability in the code is made by a real developer

This survey contains 40 code samples.

Thank you for taking the time to participate in this survey!

Now, let's start the survey

First, please tell me your background for programming.

Q1. What is your primary profession?

- ☐ Software Developer
- ☐ Student (Undergraduate)
- ☐ Student (Graduate)
- ☐ Academia (Professor or Researcher)

Q2. How many years of experience do you have with C language (in years)?

Q3. Are you familiar with the [Common Weakness Enumeration \(CWE\)](#)

- ☐ Yes
- ☐ No

Q4. Are you familiar with the [Common Vulnerabilities and Exposures \(CVE\)](#)

- ☐ Yes
- ☐ No

Continue this survey only if your answer in Q2 is >0 and you select Yes in Q3 and Q4.

Q5. Now, please record your starting time:

Date:

Time:

Code Sample 1:

```
static int vfio_msi_enable(struct vfio_pci_device *vdev, int nvec, bool msix)
{
    struct pci_dev *pdev = vdev->pdev;
    unsigned int flag = msix ? PCI_IRQ_MSIX : PCI_IRQ_MSI;
    int ret;
    if (!is_irq_none(vdev))
        return -EINVAL;
    // Using "kzalloc" to allocate memory may cause integer overflow (CWE190)
    vdev->ctx = kzalloc(nvec * sizeof(struct vfio_pci_irq_ctx), GFP_KERNEL);
    // -- vdev->ctx = kzalloc(nvec * sizeof(struct vfio_pci_irq_ctx), GFP_KERNEL);
    // ++ vdev->ctx = kcalloc(nvec, sizeof(struct vfio_pci_irq_ctx), GFP_KERNEL);
    if (!vdev->ctx)
        return -ENOMEM;
    ret = pci_alloc_irq_vectors(pdev, 1, nvec, flag);
    if (ret < nvec)
    {
        if (ret > 0)
            pci_free_irq_vectors(pdev);
        kfree(vdev->ctx);
        return ret;
    }
    vdev->num_ctx = nvec;
    vdev->irq_type = msix ? VFIO_PCI_MSIX_IRQ_INDEX : VFIO_PCI_MSI_IRQ_INDEX;
    if (!msix)
    {
        vdev->msi_qmax = fls(nvec * 2 - 1) - 1;
    }
    return 0;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 2:

```
void apm_cpu_idle()
{
    struct apmregs regs;
    if (!apminited || !apmidleon)
        return;
    // Not resetting/cleaning up registers may cause sensitive information exposure (CWE200)
    // ++ bzero(&regs, sizeof(regs));
    if (apmcall(APM_CPU_IDLE, 0, &regs) != 0)
        apm_perror("set CPU idle", &regs);
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 3:

```
int equalizer_get_parameter(effect_context_t *context, effect_param_t *p, uint32_t *size)
{
    equalizer_context_t *eq_ctxt = (equalizer_context_t *)context;
    int voffset = ((p->psize - 1) / sizeof(int32_t) + 1) * sizeof(int32_t);
    int32_t *param_tmp = (int32_t *)p->data;
    int32_t param = *param_tmp++;
    int32_t param2;
    char *name;
    void *value = p->data + voffset;
    int i;
    ALOGV("%s", __func__);
    p->status = 0;
    switch (param){
        .....
        case EQ_PARAM_CENTER_FREQ:
            ALOGV("%s: EQ_PARAM_CENTER_FREQ", __func__);
            param2 = *param_tmp;
            // Not checking the input param2 may cause sensitive information exposure (CWE200)
            // ++ if (param2 < 0 || param2 >= NUM_EQ_BANDS)
            // ++ {
            // ++     p->status = -EINVAL;
            // ++     if (param2 < 0)
            // ++     {
            // ++         android_errorWriteLog(0x534e4554, "32436341");
            // ++         ALOGW("\tERROR EQ_PARAM_CENTER_FREQ band %d", param2);
            // ++     }
            // ++     break;
            // ++ }
            *(int32_t *)value = equalizer_get_center_frequency(eq_ctxt, param2);
            break;
        .....
    }
    .....
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐ 2 ☐ 3 ☐ 4 ☐

Code Sample 4:

```
void *gdImageWebpPtrEx(gdImagePtr im, int *size, int quality)
{
    void *rv;
    gdIOCtx *out = gdNewDynamicCtx(2048, NULL);
    // Not checking the pointer may cause use of null pointer (CWE476)
    // ++ if (out == NULL)
    // ++ {
    // ++     return NULL;
    // ++ }
    if (_gdImageWebpCtx(im, out, quality))
    {
        rv = NULL;
    }
    else
    {
        rv = gdDPExtractData(out, size);
    }
    out->gd_free(out);
    return rv;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 5:

```
CURLcode Curl_close(struct Curl_easy *data)
{
    struct Curl_multi *m;
    if (!data)
        return CURLE_OK;
    Curl_expire_clear(data);
    m = data->multi;
    if (m)
        curl_multi_remove_handle(data->multi, data);
    // Not cleaning up the pointer may cause sensitive data exposure (CWE200)
    // ++ if (data->multi_easy)
    // ++{
    // ++    curl_multi_cleanup(data->multi_easy);
    // ++    data->multi_easy = NULL;
    // ++ }
    Curl_llist_destroy(&data->state.timeoutlist, NULL);
    data->magic = 0;
    if (data->state.rangestringalloc)
        free(data->state.range);
    Curl_free_request_state(data);
    Curl_ssl_close_all(data);
    Curl_safefree(data->state.first_host);
    Curl_safefree(data->state.scratch);
    Curl_ssl_free_certinfo(data);
    free(data->req.newurl);
    data->req.newurl = NULL;
    .....
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 6:

```
static int ivr_read_header(AVFormatContext *s)
{
    unsigned tag, type, len, tlen, value;
    int i, j, n, count, nb_streams = 0, ret;
    uint8_t key[256], val[256];
    AVIOContext *pb = s->pb;
    AVStream *st;
    int64_t pos, offset, temp;
    .....
    count = avio_rb32(pb);
    for (i = 0; i < count; i++)
    {
        // Not checking the input may cause excessive iteration (CWE834)
        // ++ if (avio_feof(pb))
        // ++ return AVERROR_INVALIDDATA;
        type = avio_r8(pb);
        tlen = avio_rb32(pb);
        avio_get_str(pb, tlen, key, sizeof(key));
        len = avio_rb32(pb);
        .....
    }
    .....
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 7:

```
void qemu_spice_create_host_primary(SimpleSpiceDisplay *ssd)
{
    QXLDevSurfaceCreate surface;
    uint64_t surface_size;
    // Not initializing variable may cause sensitive information exposure (CWE200)
    // ++ memset(&surface, 0, sizeof(surface));
    surface_size = (uint64_t)surface_width(ssd->ds) * surface_height(ssd->ds) * 4;
    assert(surface_size > 0);
    assert(surface_size < INT_MAX);
    if (ssd->bufsize < surface_size)
    {
        ssd->bufsize = surface_size;
        g_free(ssd->buf);
        ssd->buf = g_malloc(ssd->bufsize);
    }
    dprint(1, "%s/%d: %ux%u (size %" PRIu64 "/%d)\n", __func__, ssd->qxl.id,
    surface_width(ssd->ds), surface_height(ssd->ds), surface_size, ssd->bufsize);
    surface.format = SPICE_SURFACE_FMT_32_xRGB;
    surface.width = surface_width(ssd->ds);
    dprint(1, "%s/%d:\n", __func__, ssd->qxl.id);
    qemu_spice_destroy_primary_surface(ssd, 0, QXL_SYNC);
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 8:

```
void xf_free(xfInfo *xfi)
{
    xf_window_free(xfi);
    XCloseDisplay(xfi->display);
    // Not freeing the pointer completely may cause memory leak (CWE401)
    // ++ xfree(xfi);
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 9:

```
void dbus_test(void)
{
    char *str = getenv("DBUS_SESSION_BUS_ADDRESS");
    if (str)
    {
        char *bus = strdup(str);
        if (!bus)
            errExit("strdup");
        char *sockfile = strstr(bus, "unix:abstract=");
        if (sockfile)
        {
            sockfile += 13;
            *sockfile = '@';
            char *ptr = strchr(sockfile, ',');
            if (ptr)
                *ptr = '\\0';
            check_session_bus(sockfile);
            sockfile -= 13;
            // Freeing this pointer may cause double free (CWE415)
            // -- free(sockfile);
        }
        free(bus);
    }
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 10:

```
static int adu_release(struct inode *inode, struct file *file)
{
    struct adu_device *dev = NULL;
    int retval = 0;
    dbg(2, " %s : enter", __FUNCTION__);
    if (file == NULL)
    {
        dbg(1, " %s : file is NULL", __FUNCTION__);
        retval = -ENODEV;
        goto exit;
    }
    dev = file->private_data;
    // Not checking the pointer may cause use of null pointer (CWE476)
    // ++ if (dev == NULL)
    // ++ {
    // ++     dbg(1, " %s : object is NULL", __FUNCTION__);
    // ++     retval = -ENODEV;
    // ++     goto exit;
    // ++ }
    down(&dev->sem);
    if (dev->open_count <= 0)
    {
        dbg(1, " %s : device not opened", __FUNCTION__);
        retval = -ENODEV;
        goto exit;
    }
    retval = adu_release_internal(dev);
exit:
    if (dev)
        up(&dev->sem);
    dbg(2, " %s : leave, return value %d", __FUNCTION__, retval);
    return retval;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐ 2 ☐ 3 ☐ 4 ☐

Code Sample 11:

```
GfigObject *d_load_object(gchar *desc, FILE *fp)
{
    GfigObject *new_obj = NULL;
    gint xpnt;
    gint ypnt;
    gchar buf[MAX_LOAD_LINE];
    DobjType type;
    type = gfig_read_object_type(desc);
    .....
    while (get_line(buf, MAX_LOAD_LINE, fp, 0))
    {
        if (sscanf(buf, "%d %d", &xpnt, &ypnt) != 2)
        {
            if (!strcmp("<EXTRA>", buf))
            {
                .....
                get_line(buf, MAX_LOAD_LINE, fp, 0);
                if (sscanf(buf, "%d", &new_obj->type_data) != 1)
                {
                    g_message("Error while loading object (no type data)");
                    // Not freeing the pointer may cause memory leak (CWE401)
                    // ++ g_free(new_obj);
                    return NULL;
                }
                .....
            }
            else
                return new_obj;
        }
        .....
    }
    return new_obj;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 12:

```
static void setup(CuTest *tc)
{
    char *lensdir;
    if (asprintf(&root, "%s/build/test-save/%s", abs_top_builddir, tc->name) < 0)
    {
        CuFail(tc, "failed to set root");
    }
    if (asprintf(&lensdir, "%s/lenses", abs_top_srcdir) < 0)
        CuFail(tc, "asprintf lensdir failed");
    // Not masking may cause permission issues (CWE264)
    // ++ umask(0022);
    run(tc, "test -d %s && chmod -R u+w %s || :", root, root);
    run(tc, "rm -rf %s", root);
    run(tc, "mkdir -p %s", root);
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 13:

```
static void doXPathQuery(xmlDocPtr doc, const char *query)
{
    xmlXPathContextPtr ctxt;
    xmlXPathObjectPtr res;
    ctxt = xmlXPathNewContext(doc);
    // Not checking the pointer may cause null pointer dereference (CWE476)
    // ++ if (ctxt == NULL)
    // ++ {
    // ++     fprintf(stderr, "Out of memory for XPath\n");
    // ++     progresult = XMLLINT_ERR_MEM;
    // ++     return;
    // ++ }
    ctxt->node = (xmlNodePtr)doc;
    res = xmlXPathEval(BAD_CAST query, ctxt);
    xmlXPathFreeContext(ctxt);
    if (res == NULL)
    {
        fprintf(stderr, "XPath evaluation failure\n");
        progresult = XMLLINT_ERR_XPATH;
        return;
    }
    doXPathDump(res);
    xmlXPathFreeObject(res);
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 14:

```
int xps_true_callback_glyph_name(gs_font *pfont, gs_glyph glyph, gs_const_string
*pstr)
{
    int table_length;
    int table_offset;
    ulong format;
    int numGlyphs;
    uint glyph_name_index;
    const byte *postp;
    // Not checking the input may cause buffer overflow (CWE119)
    // ++ if (glyph >= GS_MIN_GLYPH_INDEX)
    // ++ {
    // ++     glyph -= GS_MIN_GLYPH_INDEX;
    // ++ }
    if (pfont->FontType != ft_TrueType)
    {
        pstr->size = strlen((char *)pstr->data);
        return 0;
    }
    else
    {
        return gs_throw1(-1, "glyph index %lu out of range", (ulong)glyph);
    }
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 15:

```
static int do_hidp_sock_ioctl(struct socket *sock, unsigned int cmd, void __user *argp)
{
    struct hidp_connadd_req ca;
    struct hidp_conndel_req cd;
    struct hidp_connlist_req cl;
    struct hidp_conninfo ci;
    struct socket *csock;
    struct socket *isock;
    int err;
    BT_DBG("cmd %x arg %p", cmd, argp);
    switch (cmd)
    {
    case HIDPCONNADD:
        if (!capable(CAP_NET_ADMIN))
            return -EPERM;
        if (copy_from_user(&ca, argp, sizeof(ca)))
            return -EFAULT;
        csock = sockfd_lookup(ca.ctrl_sock, &err);
        if (!csock)
            return err;
        isock = sockfd_lookup(ca.intr_sock, &err);
        if (!isock)
        {
            sockfd_put(csock);
            return err;
        }
        // Not setting the end of name may cause command injection (CWE77)
        // ++ ca.name[sizeof(ca.name) - 1] = 0;
        err = hidp_connection_add(&ca, csock, isock);
        .....
    }
    return -EINVAL;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 16:

```
static void ffmnal_stop_decoder(AVCodecContext *avctx)
{
    MMALDecodeContext *ctx = avctx->priv_data;
    MMAL_COMPONENT_T *decoder = ctx->decoder;
    MMAL_BUFFER_HEADER_T *buffer;
    mmal_port_disable(decoder->input[0]);
    mmal_port_disable(decoder->output[0]);
    mmal_port_disable(decoder->control);
    mmal_port_flush(decoder->input[0]);
    mmal_port_flush(decoder->output[0]);
    mmal_port_flush(decoder->control);
    while ((buffer = mmal_queue_get(ctx->queue_decoded_frames)))
        mmal_buffer_header_release(buffer);
    while (ctx->waiting_buffers)
    {
        FFBufferEntry *buffer = ctx->waiting_buffers;
        ctx->waiting_buffers = buffer->next;
        av_buffer_unref(&buffer->ref);
        // Not freeing the buffer may cause memory leak (CWE401)
        // ++ av_free(buffer);
    }
    ctx->waiting_buffers_tail = NULL;
    assert(avpriv_atomic_get(&ctx->packets_buffered) == 0);
    ctx->frames_output = ctx->eos_received = ctx->eos_sent = ctx->packets_sent =
    ctx->extradata_sent = 0;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐ 2 ☐ 3 ☐ 4 ☐

Code Sample 17:

```
static int generate_subkeys(mbedtls_cmac_context *ctx)
{
    int ret;
    unsigned char L[16];
    size_t olen;
    memset(L, 0, 16);
    if ((ret = mbedtls_cipher_update(&ctx->cipher_ctx, L, 16, L, &olen)) != 0)
    {
        return (ret);
    }
    multiply_by_u(ctx->K1, L);
    multiply_by_u(ctx->K2, ctx->K1);
    // Not resetting L to zero may cause sensitive information exposure (CWE200)
    // ++ mbedtls_zeroize(L, sizeof(L));
    return (0);
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 18:

```
static VTermScreen *screen_new(VTerm *vt)
{
    VTermState *state = vterm_obtain_state(vt);
    VTermScreen *screen;
    int rows, cols;
    if (state == NULL)
        return NULL;
    screen = vterm_allocator_malloc(vt, sizeof(VTermScreen));
    // Not checking the pointer may cause null pointer dereference
    // ++ if (screen == NULL)
    // ++ return NULL;
    vterm_get_size(vt, &rows, &cols);
    screen->vt = vt;
    screen->state = state;
    screen->damage_merge = VTERM_DAMAGE_CELL;
    screen->damaged.start_row = -1;
    screen->pending_scrollrect.start_row = -1;
    screen->rows = rows;
    screen->cols = cols;
    screen->callbacks = NULL;
    screen->cbdata = NULL;
    screen->buffers[0] = realloc_buffer(screen, NULL, rows, cols);
    screen->buffer = screen->buffers[0];
    screen->sb_buffer = vterm_allocator_malloc(screen->vt,
sizeof(VTermScreenCell) * cols);
    if (screen->buffer == NULL || screen->sb_buffer == NULL)
    {
        vterm_screen_free(screen);
        return NULL;
    }
    vterm_state_set_callbacks(screen->state, &state_cbs, screen);
    return screen;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐ 2 ☐ 3 ☐ 4 ☐

Code Sample 19:

```
static struct glink_channel *qcom_glink_alloc_channel(struct qcom_glink *glink,
const char *name)
{
    struct glink_channel *channel;
    channel = kzalloc(sizeof(*channel), GFP_KERNEL);
    // Not checking the pointer may cause null pointer dereference
    // ++ IF (!channel)
    // ++ return ERR_PTR(-ENOMEM);
    spin_lock_init(&channel->recv_lock);
    spin_lock_init(&channel->intent_lock);
    channel->glink = glink;
    channel->name = kstrdup(name, GFP_KERNEL);
    init_completion(&channel->open_req);
    init_completion(&channel->open_ack);
    init_completion(&channel->intent_req_comp);
    INIT_LIST_HEAD(&channel->done_intents);
    INIT_WORK(&channel->intent_work, qcom_glink_rx_done_work);
    idr_init(&channel->liids);
    idr_init(&channel->riids);
    kref_init(&channel->refcount);
    return channel;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 20:

```
BOOL license_read_scope_list(wStream *s, SCOPE_LIST *scopeList)
{
    UINT32 i;
    UINT32 scopeCount;
    if (Stream_GetRemainingLength(s) < 4)
        return FALSE;
    Stream_Read_UINT32(s, scopeCount);
    // Not checking the input may cause numeric error (CWE189)
    // ++ if (Stream_GetRemainingLength(s) / sizeof(LICENSE_BLOB) < scopeCount)
    // ++ return FALSE;
    scopeList->count = scopeCount;
    scopeList->array = (LICENSE_BLOB *)malloc(sizeof(LICENSE_BLOB) * scopeCount);
    for (i = 0; i < scopeCount; i++)
    {
        scopeList->array[i].type = BB_SCOPE_BLOB;
        if (!license_read_binary_blob(s, &scopeList->array[i]))
            return FALSE;
    }
    return TRUE;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 21:

```
static void aoedev_freedevice(struct aoedev *d)
{
    if (d->gd)
    {
        aoedisk_rm_sysfs(d);
        del_gendisk(d->gd);
        put_disk(d->gd);
    }
    kfree(d->frames);
    // Not destroying the buf may cause sensitive information exposure (CWE200)
    // ++ if (d->bufpool)
    // ++ mempool_destroy(d->bufpool);
    kfree(d);
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 22:

```
void destroy_list(int list_id)
{
    ds_set_t *sp = NULL;
    ds_set_t *sp1 = NULL;
    ds_dest_t *dest = NULL;
    sp = ds_lists[list_id];
    while (sp)
    {
        sp1 = sp->next;
        for (dest = sp->dlist; dest != NULL; dest = dest->next)
        {
            if (dest->uri.s != NULL)
            {
                shm_free(dest->uri.s);
                dest->uri.s = NULL;
            }
        }
        // Not freeing the pointer may cause memory leak (CWE401)
        // ++ if (sp->dlist != NULL)
        // ++     shm_free(sp->dlist);
        shm_free(sp);
        sp = sp1;
    }
    ds_lists[list_id] = NULL;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐ 2 ☐ 3 ☐ 4 ☐

Code Sample 23:

```
static int patch_cs4213(struct hda_codec *codec)
{
    struct cs_spec *spec;
    int err;
    spec = kzalloc(sizeof(*spec), GFP_KERNEL);
    // Not checking the pointer may cause null pointer dereference (CWE476)
    // ++ if (!spec)
    // ++ return -ENOMEM;
    codec->spec = spec;
    spec->vendor_nid = CS4213_VENDOR_NID;
    err = cs421x_parse_auto_config(codec);
    if (err < 0)
        goto error;
    codec->patch_ops = cs421x_patch_ops;
    return 0;
error:
    cs_free(codec);
    codec->spec = NULL;
    return err;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 24:

```
static int caif_seqpkt_recvmmsg(struct kiocb *iocb, struct socket *sock, struct msghdr
*m, size_t len, int flags)
{
    struct sock *sk = sock->sk;
    struct sk_buff *skb;
    int ret;
    int copylen;
    ret = -EOPNOTSUPP;
    if (m->msg_flags & MSG_OOB)
        goto read_error;
    // Not initializing variable may cause sensitive information exposure (CWE200)
    // ++ m->msg_namelen = 0;
    skb = skb_recv_datagram(sk, flags, 0, &ret);
    if (!skb)
        goto read_error;
    copylen = skb->len;
    if (len < copylen)
    {
        m->msg_flags |= MSG_TRUNC;
        copylen = len;
    }
    ret = skb_copy_datagram_iovec(skb, 0, m->msg_iov, copylen);
    if (ret)
        goto out_free;
    ret = (flags & MSG_TRUNC) ? skb->len : copylen;
out_free:
    skb_free_datagram(sk, skb);
    caif_check_flow_release(sk);
    return ret;
read_error:
    return ret;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 25:

```
static ssize_t stellaris_enet_receive(NetClientState *nc, const uint8_t *buf,
size_t size)
{
    stellaris_enet_state *s = qemu_get_nic_opaque(nc);
    int n;
    uint8_t *p;
    uint32_t crc;
    if ((s->rctl & SE_RCTL_RXEN) == 0)
        return -1;
    if (s->np >= 31)
    {
        return 0;
    }
    DPRINTF("Received packet len=%zu\n", size);
    n = s->next_packet + s->np;
    if (n >= 31)
        n -= 31;
    // Not checking the input data size properly (CWE 20)
    // ++ if (size >= sizeof(s->rx[n].data) - 6)
    // ++ {
    // ++     s->ris |= SE_INT_FOV;
    // ++     stellaris_enet_update(s);
    // ++     return -1;
    // ++ }
    s->np++;
    s->rx[n].len = size + 6;
    p = s->rx[n].data;
    *(p++) = (size + 6);
    memset(p, 0, (6 - size) & 3);
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐ 2 ☐ 3 ☐ 4 ☐

Code Sample 26:

```
static int ext4_split_unwritten_extents(handle_t *handle, struct inode *inode,
struct ext4_map_blocks *map, struct ext4_ext_path *path, int flags)
{
    ext4_lblk_t eof_block;
    ext4_lblk_t ee_block;
    struct ext4_extent *ex;
    unsigned int ee_len;
    int split_flag = 0, depth;
    ext_debug("ext4_split_unwritten_extents: inode %lu, logical"
              "block %llu, max_blocks %u\n",
              inode->i_ino, (unsigned long long)map->m_lblk, map->m_len);
    eof_block = (inode->i_size + inode->i_sb->s_blocksize - 1) >>
inode->i_sb->s_blocksize_bits;
    if (eof_block < map->m_lblk + map->m_len)
        eof_block = map->m_lblk + map->m_len;
    depth = ext_depth(inode);
    ex = path[depth].p_ext;
    ee_block = le32_to_cpu(ex->ee_block);
    ee_len = ext4_ext_get_actual_len(ex);
    split_flag |= ee_block + ee_len <= eof_block ? EXT4_EXT_MAY_ZEROOUT : 0;
    split_flag |= EXT4_EXT_MARK_UNINIT2;
    // Not checking the flag may cause race condition (CWE362)
    // ++ if (flags & EXT4_GET_BLOCKS_CONVERT)
    // ++ split_flag |= EXT4_EXT_DATA_VALID2;
    flags |= EXT4_GET_BLOCKS_PRE_IO;
    return ext4_split_extent(handle, inode, path, map, split_flag, flags);
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 27:

```
static int vmci_transport_dgram_dequeue(struct kiocb *kiocb, struct vsock_sock
*vsk, struct msghdr *msg, size_t len, int flags)
{
    int err;
    int noblock;
    struct vmci_datagram *dg;
    size_t payload_len;
    struct sk_buff *skb;
    noblock = flags & MSG_DONTWAIT;
    if (flags & MSG_OOB || flags & MSG_ERRQUEUE)
        return -EOPNOTSUPP;
    // Not setting the variable may cause sensitive information exposure (CWE200)
    // ++ msg->msg_namelen = 0;
    err = 0;
    skb = skb_recv_datagram(&vsk->sk, flags, noblock, &err);
    .....
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐ 2 ☐ 3 ☐ 4 ☐

Code Sample 28:

```
static int inotify_release(struct inode *ignored, struct file *file)
{
    struct fsnotify_group *group = file->private_data;
    // The declaration of variable may cause resource error (CWE399)
    // -- struct user_struct *user = group->inotify_data.user;
    pr_debug("%s: group=%p\n", __func__, group);
    fsnotify_clear_marks_by_group(group);
    fsnotify_put_group(group);
    atomic_dec(&user->inotify_devs);
    return 0;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 29:

```
static void rt6_free_pcpu(struct rt6_info *non_pcpu_rt)
{
    int cpu;
    if (!non_pcpu_rt->rt6i_pcpu)
        return;
    for_each_possible_cpu(cpu)
    {
        struct rt6_info **ppcpu_rt;
        struct rt6_info *pcpu_rt;
        ppcpu_rt = per_cpu_ptr(non_pcpu_rt->rt6i_pcpu, cpu);
        pcpu_rt = *ppcpu_rt;
        if (pcpu_rt)
        {
            rt6_rcu_free(pcpu_rt);
            *ppcpu_rt = NULL;
        }
    }
    // Not freeing the pointer may cause memory leak (CWE401)
    // ++ free_percpu(non_pcpu_rt->rt6i_pcpu);
    non_pcpu_rt->rt6i_pcpu = NULL;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐ 2 ☐ 3 ☐ 4 ☐

Code Sample 30:

```
static int atalk_getname(struct socket *sock, struct sockaddr *uaddr, int *uaddr_len, int
peer)
{
    struct sockaddr_at sat;
    struct sock *sk = sock->sk;
    struct atalk_sock *at = at_sk(sk);
    if (sock_flag(sk, SOCK_ZAPPED))
        if (atalk_autobind(sk) < 0)
            return -ENOBUFS;
    *uaddr_len = sizeof(struct sockaddr_at);
    // Not setting the variable may cause sensitive information exposure (CWE200)
    // ++ memset(&sat.sat_zero, 0, sizeof(sat.sat_zero));
    if (peer)
    {
        if (sk->sk_state != TCP_ESTABLISHED)
            return -ENOTCONN;
        sat.sat_addr.s_net = at->dest_net;
        sat.sat_addr.s_node = at->dest_node;
        sat.sat_port = at->dest_port;
    }
    else
    {
        sat.sat_addr.s_net = at->src_net;
        sat.sat_addr.s_node = at->src_node;
        sat.sat_port = at->src_port;
    }
    sat.sat_family = AF_APPLETALK;
    memcpy(uaddr, &sat, sizeof(sat));
    return 0;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 31:

```
static int r5l_recovery_log(struct r5l_log *log)
{
    struct r5l_recovery_ctx ctx;
    ctx.pos = log->last_checkpoint;
    ctx.seq = log->last_cp_seq;
    ctx.meta_page = alloc_page(GFP_KERNEL);
    // Not checking the pointer may cause null pointer dereference (CWE476)
    // ++ if (!ctx.meta_page)
    // ++ return -ENOMEM;
    r5l_recovery_flush_log(log, &ctx);
    __free_page(ctx.meta_page);
    if (ctx.seq > log->last_cp_seq + 1)
    {
        int ret;
        ret = r5l_log_write_empty_meta_block(log, ctx.pos, ctx.seq + 10);
        if (ret)
            return ret;
        log->seq = ctx.seq + 11;
        log->log_start = r5l_ring_add(log, ctx.pos, BLOCK_SECTORS);
        r5l_write_super(log, ctx.pos);
        log->last_checkpoint = ctx.pos;
        log->next_checkpoint = ctx.pos;
    }
    else
    {
        log->log_start = ctx.pos;
        log->seq = ctx.seq;
    }
    return 0;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 32:

```
static int name_len(netdissect_options *ndo, const unsigned char *s, const unsigned char
*maxbuf)
{
    const unsigned char *s0 = s;
    unsigned char c;
    if (s >= maxbuf)
        return (-1);
    ND_TCHECK2(*s, 1);
    c = *s;
    if ((c & 0xC0) == 0xC0)
        return (2);
    while (*s)
    {
        if (s >= maxbuf)
            return (-1);
        ND_TCHECK2(*s, 1);
        s += (*s) + 1;
        // Not checking the pointer may cause buffer overflow (CWE119)
        // ++ ND_TCHECK2(*s, 1);
    }
    return (PTR_DIFF(s, s0) + 1);
trunc:
    return (-1);
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐ 2 ☐ 3 ☐ 4 ☐

Code Sample 33:

```
static int read_packet(AVFormatContext *s, AVPacket *pkt){
    JVDemuxContext *jv = s->priv_data;
    AVIOContext *pb = s->pb;
    AVStream *ast = s->streams[0];
    .....
    switch (jv->state){
    case JV_VIDEO:
        jv->state++;
        // Not initializing the buffer may cause sensitive information exposure (CWE200)
        // ++ memset(pkt->data + JV_PREAMBLE_SIZE + size, 0, FF_INPUT_BUFFER_PADDING_SIZE);
        pkt->size = size + JV_PREAMBLE_SIZE;
        pkt->stream_index = 1;
        pkt->pts = jv->pts;
        .....
    }
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐ 2 ☐ 3 ☐ 4 ☐

Code Sample 34:

```
sctp_disposition_t sctp_sf_beat_8_3(const struct sctp_endpoint *ep, const struct
sctp_association *asoc, const sctp_subtype_t type, void *arg, sctp_cmd_seq_t *commands)
{
    struct sctp_chunk *chunk = arg;
    struct sctp_chunk *reply;
    // Not initializing variable may cause sensitive information exposure (CWE200)
    // ++ paylen = 0;
    if (!sctp_vtag_verify(chunk, asoc))
        return sctp_sf_pdiscard(ep, asoc, type, arg, commands);
    if (!sctp_chunk_length_valid(chunk, sizeof(sctp_heartbeat_chunk_t)))
        return sctp_sf_violation_chunklen(ep, asoc, type, arg, commands);
    chunk->subh.hb_hdr = (sctp_heartbeat_hdr_t *)chunk->skb->data;
    paylen = ntohs(chunk->chunk_hdr->length) - sizeof(sctp_chunkhdr_t);
    if (!pskb_pull(chunk->skb, paylen))
        goto nomem;
    reply = sctp_make_heartbeat_ack(asoc, chunk, chunk->subh.hb_hdr, paylen);
    if (!reply)
        goto nomem;
    sctp_add_cmd_sf(commands, Sctp_CMD_REPLY, Sctp_CHUNK(reply));
    return Sctp_DISPOSITION_CONSUME;
nomem:
    return Sctp_DISPOSITION_NOMEM;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 35:

```
void vhost_dev_stop(struct vhost_dev *hdev, VirtIODevice *vdev)
{
    int i, r;
    for (i = 0; i < hdev->nvqs; ++i)
    {
        vhost_virtqueue_cleanup(hdev, vdev, hdev->vqs + i, i);
    }
    vhost_client_sync_dirty_bitmap(&hdev->client, 0,
(target_phys_addr_t)~0x0ull);
    r = vdev->binding->set_guest_notifiers(vdev->binding_opaque, false);
    if (r < 0)
    {
        fprintf(stderr, "vhost guest notifier cleanup failed: %d\n", r);
        fflush(stderr);
    }
    assert(r >= 0);
    hdev->started = false;
    qemu_free(hdev->log);
    // Not setting the pointer may cause use after free (CWE416)
    // ++ hdev->log = NULL;
    hdev->log_size = 0;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 36:

```
void hists__decay_entries(struct hists *hists)
{
    struct rb_node *next = rb_first(&hists->entries);
    struct hist_entry *n;
    while (next)
    {
        n = rb_entry(next, struct hist_entry, rb_node);
        next = rb_next(&n->rb_node);
        if (hists__decay_entry(hists, n) && !n->used)
        {
            rb_erase(&n->rb_node, &hists->entries);
            if (sort__need_collapse)
                rb_erase(&n->rb_node_in, &hists->entries_collapsed);
            // Not freeing the pointer may cause memory leak (CWE401)
            // ++ hist_entry_free(n);
            --hists->nr_entries;
        }
    }
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 37:

```
int dev_forward_skb(struct net_device *dev, struct sk_buff *skb)
{
    skb_orphan(skb);
    if (!(dev->flags & IFF_UP) || (skb->len > (dev->mtu + dev->hard_header_len)))
    {
        // Not freeing the pointer may cause memory leak (CWE401)
        // ++ kfree_skb(skb);
        return NET_RX_DROP;
    }
    skb_set_dev(skb, dev);
    skb->tstamp.tv64 = 0;
    skb->pkt_type = PACKET_HOST;
    skb->protocol = eth_type_trans(skb, dev);
    return netif_rx(skb);
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 38:

```
static void wmem_block_split_used_chunk(wmem_block_allocator_t *allocator,
wmem_block_chunk_t *chunk, const size_t size)
{
    wmem_block_chunk_t *extra;
    size_t aligned_size, available;
    gboolean last;
    g_assert(chunk->used);
    // Not checking the size may cause buffer overflow (CWE119)
    // ++ g_assert(WMEM_CHUNK_DATA_LEN(chunk) >= size);
    aligned_size = WMEM_ALIGN_SIZE(size);
    if (aligned_size + sizeof(wmem_block_chunk_t) > WMEM_CHUNK_DATA_LEN(chunk))
    {
        return;
    }
    last = chunk->last;
    available = chunk->len;
    chunk->len = (guint32)(aligned_size + sizeof(wmem_block_chunk_t));
    chunk->last = FALSE;
    extra = WMEM_CHUNK_NEXT(chunk);
    available -= (aligned_size + sizeof(wmem_block_chunk_t));
    extra->len = (guint32)available;
    extra->last = last;
    extra->prev = (guint32)(aligned_size + sizeof(wmem_block_chunk_t));
    extra->used = FALSE;
    wmem_block_add_to_free_list(allocator, extra);
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 39:

```
static gboolean user_set_icon_file(AccountsUser *auser, GDBusMethodInvocation
*context, const gchar *filename)
{
    User *user = (User *)auser;
    int uid;
    const gchar *action_id;
    if (!get_caller_uid(context, &uid))
    {
        throw_error(context, ERROR_FAILED, "identifying caller failed");
        return FALSE;
    }
    if (user->uid == (uid_t)uid)
        action_id = "org.freedesktop.accounts.change-own-user-data";
    else
        action_id = "org.freedesktop.accounts.user-administration";
    // Not checking the authorization may cause permission issue (CWE264)
    // ++ daemon_local_check_auth(user->daemon, user, action_id, TRUE,
user_change_icon_file_authorized_cb, context, g_strdup(filename),
(GDestroyNotify)g_free);
    return TRUE;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐

2 ☐

3 ☐

4 ☐

Code Sample 40:

```
int av_image_fill_linesizes(int linesizes[4], enum PixelFormat pix_fmt, int width){
    int i;
    const AVPixFmtDescriptor *desc = &av_pix_fmt_descriptors[pix_fmt];
    int max_step[4];
    int max_step_comp[4];
    memset(linesizes, 0, 4 * sizeof(linesizes[0]));
    if ((unsigned)pix_fmt >= PIX_FMT_NB || desc->flags & PIX_FMT_HWACCEL)
        return AERROR(EINVAL);
    if (desc->flags & PIX_FMT_BITSTREAM) {
        // Not checking the input width properly (CWE20)
        // ++ if (width > (INT_MAX - 7) / (desc->comp[0].step_minus1 + 1))
        // ++ return AERROR(EINVAL);
        linesizes[0] = (width * (desc->comp[0].step_minus1 + 1) + 7) >> 3;
        return 0;
    }
    .....
    return 0;
}
```

How confident do you think the vulnerability is made by a real developer rather than being synthesized (by an automated code editing tool)?

1 ☐ 2 ☐ 3 ☐ 4 ☐

Thank you very much for your help

Before leaving, please record your finishing time:

Date:

Time: